Learning to Avoid the Cliff

SARSA in CliffWalking-v1

Eman Fayyaz | FA23-BAI-030
Topics in AI - Assignment 1
COMSATS University Islamabad, Attock Campus | 20 October 2026

1. Problem description

The agent must navigate a 4-by-12 grid from the bottom-left start to the bottom-right goal while avoiding a cliff along the intervening bottom-row cells. The experiment uses the non-slippery version: movement is deterministic and every episode starts at the same location. The objective is to maximize cumulative reward, balancing route length against costly falls.

Element	Representation
States	Integer s = 12 x row + column; the declared observation space has 48 entries. Start is 36 and terminal goal is 47. Cliff cells immediately return the agent to start.
Actions	0 = up; 1 = right; 2 = down; 3 = left.
Rewards	-1 for each ordinary move, including arrival at the goal; -100 for a cliff fall. No extra positive goal reward is added.
Ending	Reaching the goal terminates the episode. A project-defined 500-step limit truncates long rollouts; falling does not terminate.

2. Methodology and architecture

SARSA is an on-policy, model-free temporal-difference algorithm. A 48-by-4 Q-table starts at zero. Epsilon-greedy selection chooses a uniformly random action with probability epsilon and otherwise chooses a maximal-Q action. Ties are random during training; evaluation uses deterministic argmax.

Update: Q(s,a) ← Q(s,a) + alpha [y - Q(s,a)]
Target: y = r if terminated; otherwise y = r + gamma Q(s',a').

The next action a' is sampled from the same behavior policy before the update and is carried into the next step. This distinguishes SARSA from Q-learning, whose target takes a maximum over next actions. At a time-limit truncation the target still bootstraps; at the actual goal it does not.

Implementation: agent.py defines SARSA; utils.py creates the environment, runs training/evaluation and saves plots/GIFs; main.py coordinates the experiments. The executed notebook calls the same functions. Gymnasium supplies the environment, while NumPy implements the learning algorithm without a pretrained RL library.

3. Experimental setup and hyperparameters

Three constant training exploration rates (0.01, 0.1 and 0.3) are compared using seeds 30, 31 and 32, each for 3,000 episodes. Training resets use seed + episode; a separate NumPy generator controls action sampling. The deterministic start does not vary with the reset seed. The experiment comprises nine complete training runs.

Every 500 episodes, each policy is evaluated on 100 episodes with a common evaluation epsilon of 0.1 and action-generator seed 10000. This common exploration rate measures route robustness on the same basis. The selected training epsilon maximizes final validation mean return averaged across all three training seeds. The seed-30 model for that configuration is retained; the best individual seed is not selected.

Hyperparameter	Final setting
Environment	CliffWalking-v1; is_slippery=False
Algorithm / Q-table	SARSA / 48 x 4, initialized to zero
Learning rate alpha	0.5
Discount factor gamma	1.0 (undiscounted episodic return)
Selected constant training epsilon	0.1
Budget and seeds	3,000 episodes per run; seeds 30, 31, 32
Maximum episode length	500 steps, explicitly set by the project
Validation	100 episodes every 500 training episodes; epsilon=0.1
Final evaluation	300 episodes per policy; action-generator seed 20000

4. Hyperparameter sensitivity

Training epsilon	Final validation return: mean +/- SD	Cliff falls / episode
0.01	-25.53 +/- 3.93	0.063
0.1	-22.80 +/- 1.31	0.027
0.3	-92.26 +/- 100.27	0.063

Table 1. Validation uses epsilon=0.1 for every candidate. SD is the sample standard deviation of the three run-level mean returns; it is not an episode-level confidence interval. Only training epsilon is tuned. Alpha and gamma remain fixed, so no claim is made that all hyperparameters are globally optimal.

5. Results and visualizations

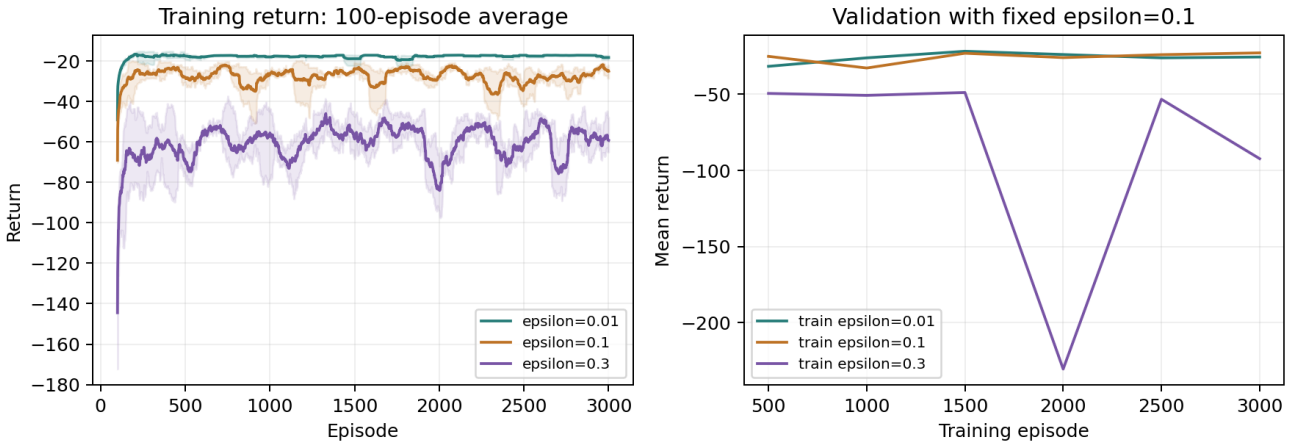


Figure 1. Left: mean smoothed training return, with minimum-to-maximum shading across three seeds. Right: mean validation return under the common exploratory evaluation policy. Larger (less negative) returns are better.

Policy	Success	Return	Steps	Falls
Uniform random	8.7%	-4926.20	482.09	44.890
SARSA, greedy	100.0%	-17.00	17.00	0.000
SARSA, epsilon=0.1	100.0%	-25.15	20.53	0.047

Table 2. Final evaluation: 300 episodes per policy, without learning. Return, steps and falls are episode means, including truncated episodes. Success means reaching the goal before the cap. The exploratory and random evaluations use a fresh action-generator seed of 20000, distinct from validation.

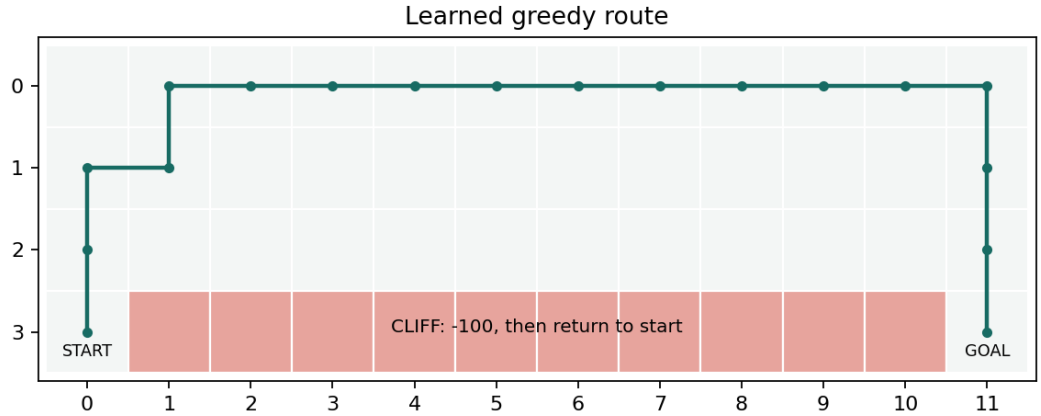


Figure 2. The saved model reaches the goal in 17 greedy steps with return -17 and 0 cliff falls. The accompanying trained_agent.gif animates this route.

Because the environment has a fixed start, deterministic transitions and deterministic greedy evaluation, repeated greedy episodes reproduce the same route. The 300 greedy successes demonstrate consistent execution of that route rather than generalization to 300 distinct tasks. Random and epsilon-greedy actions create varied trajectories.

6. Discussion and failure modes

The selected training epsilon is 0.1. Its saved policy achieves 100.0% greedy success, with mean return -17.00 and 17.00 steps. Under epsilon=0.1 evaluation, mean return changes to -25.15 and mean cliff falls to 0.047. The random baseline achieves 8.7% success and mean return -4926.20.

The epsilon=0.3 configuration has the weakest final validation mean (-92.26) and largest across-seed SD (100.27), whereas epsilon=0.1 gives -22.80 +/- 1.31. This supports epsilon=0.1 for this training budget; it does not imply that more exploration always performs worse.

Why exploration matters. SARSA learns from the next action actually sampled by the behavior policy, including random moves. A path close to the cliff can be short under greedy execution but risky when actions remain exploratory. The validation comparison uses a fixed epsilon so that candidate policies are assessed under the same level of action randomness. The route plot provides direct evidence of the selected policy; a safer route is not assumed solely from the algorithm name.

Observed sources of poor return. Cliff falls produce a large penalty and send the agent back to the start, adding further moves. Boundary actions may leave the agent in the same cell while still costing one reward. An incompletely learned policy can loop until the 500-step cap. These mechanisms explain why reaching the goal eventually does not necessarily imply a good return.

Learning stability. Training return is noisy because exploration remains constant throughout each run. The smoothed curves show empirical behavior, not a proof of convergence. A fixed positive learning rate and constant exploration do not satisfy the standard diminishing-step-size/greedy-in-the-limit convergence conditions. Epsilon=0 at evaluation also changes the policy relative to the policy SARSA learned during training.

Limits of the experiment. The comparison uses three training seeds and tunes only epsilon. All policies are evaluated on one fixed map. Reset-seed separation does not produce new maps or starting states. Reported results do not establish transfer to slippery dynamics, larger state spaces or real navigation tasks. A Q-table would become impractical as the state space grows.

7. Conclusion

A from-scratch SARSA agent learns to reach the goal in CliffWalking while accounting for costly cliff falls. The experiment shows how the training exploration rate affects learning and performance under continued exploration. The saved Q-table, executed notebook, raw training histories, plots and successful GIF make the implementation and measured results reproducible.

References

Farama Foundation. Cliff Walking, Gymnasium 1.2.3 documentation. https://gymnasium.farama.org/v1.2.3/environments/toy_text/cliff_walking/

Sutton, R. S. and Barto, A. G. Reinforcement Learning: An Introduction, 2nd edition, 2018. Chapter 6: SARSA and the cliff-walking example.